

10주 차 보고서

(2026년 3 월 01 일 ~ 2026년 3 월 07 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

```
pushd %~dp0
protoc.exe -I=./ --cpp_out=./ ./Enum.proto
protoc.exe -I=./ --cpp_out=./ ./Struct.proto
protoc.exe -I=./ --cpp_out=./ ./Protocol.proto

GenPackets.exe --path=./Protocol.proto --output=ClientPacketHandler --recv=C_ --send=S_
GenPackets.exe --path=./Protocol.proto --output=ServerPacketHandler --recv=S_ --send=C_

IF ERRORLEVEL 1 PAUSE

XCOPY /Y Enum.pb.h "../../../GameServer"
XCOPY /Y Enum.pb.cc "../../../GameServer"
XCOPY /Y Struct.pb.h "../../../GameServer"
XCOPY /Y Struct.pb.cc "../../../GameServer"
XCOPY /Y Protocol.pb.h "../../../GameServer"
XCOPY /Y Protocol.pb.cc "../../../GameServer"
XCOPY /Y ClientPacketHandler.h "../../../GameServer"

XCOPY /Y Enum.pb.h "../../../DummyClient"
XCOPY /Y Enum.pb.cc "../../../DummyClient"
XCOPY /Y Struct.pb.h "../../../DummyClient"
XCOPY /Y Struct.pb.cc "../../../DummyClient"
XCOPY /Y Protocol.pb.h "../../../DummyClient"
XCOPY /Y Protocol.pb.cc "../../../DummyClient"
XCOPY /Y ServerPacketHandler.h "../../../DummyClient"

DEL /Q /F *.pb.h
DEL /Q /F *.pb.cc
DEL /Q /F *.h

PAUSE
```

```

import argparse
import Jinja2
import ProtoParser

def main():

    arg_parser = argparse.ArgumentParser(description = 'PacketGenerator')
    arg_parser.add_argument('--path', type=str, default='C:/Users/0412j/Server Part4/Server/Common/Protobuf/bin/Protocol.proto', help='proto path')
    arg_parser.add_argument('--output', type=str, default='TestPacketHandler', help='output file')
    arg_parser.add_argument('--recv', type=str, default='C_', help='recv convention')
    arg_parser.add_argument('--send', type=str, default='S_', help='send convention')
    args = arg_parser.parse_args()

    parser = ProtoParser.ProtoParser(1000, args.recv, args.send)
    parser.parse_proto(args.path)
    file_loader = Jinja2.FileSystemLoader('Templates')
    env = Jinja2.Environment(loader=file_loader)

    template = env.get_template('PacketHandler.h')
    output = template.render(parser=parser, output=args.output)

    f = open(args.output+'.h', 'w+')
    f.write(output)
    f.close()

    print(output)
    return

if __name__ == '__main__':
    main()

```

파일을 생성해주는 배치파일과, 패킷을 자동적으로 만들어주는 파이썬파일을 만들었다. 배치파일을 실행시키면 구글 protobuf에 필요한 Enum.pb.h / Enum.pb.h / Enum.pb.cc 등등을 기존에 있던 것을 삭제하고 새롭게 필요한 부분으로 생성해준다.

```

syntax = "proto3";
package Protocol;

enum PlayerType
{
    PLAYER_TYPE_NONE = 0;
    PLAYER_TYPE_KNIGHT = 1;
    PLAYER_TYPE_MAGE = 2;
    PLAYER_TYPE_ARCHER = 3;
}

```

```

syntax = "proto3";
package Protocol;

import "Enum.proto";
import "Struct.proto";

message C_LOGIN
{
}

message S_LOGIN
{
    bool success = 1;
    repeated Player players = 2; // 아이디 발급 전
}

message C_ENTER_GAME
{
    uint64 playerId = 1;
}

message S_ENTER_GAME
{
    bool success = 1;
}

message C_CHAT
{
    string msg = 1;
}

message S_CHAT
{
    uint64 playerId = 1;
    string msg = 2;
}

```

```

syntax = "proto3";
package Protocol;

import "Enum.proto";

message Player
{
    uint64 id = 1;
    string name = 2;
    PlayerType playerType = 3;
    // 스탯 정보...
}

```

```

class ProtoParser():
    def __init__(self, start_id, recv_prefix, send_prefix):
        self.recv_pkt = [] # 수신 패킷 목록
        self.send_pkt = [] # 송신 패킷 목록
        self.total_pkt = [] # 모든 패킷 목록
        self.start_id = start_id
        self.id = start_id
        self.recv_prefix = recv_prefix
        self.send_prefix = send_prefix

    def parse_proto(self, path):
        f = open(path, 'r')
        lines = f.readlines()

        for line in lines:
            if line.startswith('message') == False:
                continue

            pkt_name = line.split()[1].upper()
            if pkt_name.startswith(self.recv_prefix):
                self.recv_pkt.append(Packet(pkt_name, self.id))
            elif pkt_name.startswith(self.send_prefix):
                self.send_pkt.append(Packet(pkt_name, self.id))
            else:
                continue

            self.total_pkt.append(Packet(pkt_name, self.id))
            self.id += 1

        f.close()

class Packet:
    def __init__(self, name, id):
        self.name = name
        self.id = id

```

프로토 파일들을 읽을 수 있는 데이터 파서 역할을 할 수 있게 만들어주었다. 서버와 클라의 프로토콜을 자동화하기 위한 핵심 역할이다.

```
bool Handle_S_CHAT(PacketSessionRef& session, Protocol::S_CHAT& pkt)
{
    std::cout << pkt.msg() << endl;
    return true;
}
```

기존에 있던 서버 패킷 핸들러와 클라이언트 패킷 핸들러를 이용하여 간단한 채팅 시스템을 만들었다.

다음 주 할 일

여태까지 만든 서버 프레임워크로 클라이언트에 붙여볼 예정이다. 구글 protobuf를 이용하기 때문에 클라이언트에서도 구글 protobuf를 인식할 수 있게 수정을 할 것이다.